CS1783 Artificial Intelligence Programming
Schedule GA Analysis
October 1st 2026
Jacob Shultz

1. Introduction

Overview

In this assignment we are tasked with applying a genetic algorithm to the problem of scheduling university courses. In this paper we will discuss the background and implementation of this problem. Lastly, results will be analyzed.

Background

Based on Charles Darwin's theory of evolution, a genetic algorithm simulates the evolutionary cycle by implementing crossover, selection, and mutation. Selection refers to the most "fit" mates reproducing. Crossover refers to the genes of two parents combining to create the child. Mutation refers to a random manipulation of a gene which occurs in nature occasionally (Kramer, 2017. p 5).

In this problem we are tasked with optimizing a university course schedule. A schedule consists of timeslots (or periods), rooms, courses, and professors.

Ignoring possible logical restrictions for a schedule (such as a professor cannot teach multiple courses in the same timeslot) we can use the following formula to calculate the number of combinations of courses, rooms, and timeslots.

```
T = number of timeslots
R = number of rooms
C = number of courses

Combinations = (T * R)! / (C!((T * R) – C)!)
```

In our assignment the problem scope was T = 7, R = 7, C = 26. Therefore, before adding restrictions, there are 58,343,356,817,424 possible combinations of timeslots, rooms, and courses.

Combining the enormous solution space with logical restrictions and preferred characteristics quickly makes this an impossible problem to optimize with typical programming approaches. Perhaps a passing solution could be found, but a close-to-optimal solution would be unlikely.

In his text, Kramer describes the usefulness of genetic algorithms for problems like this by describing the problem of the traveling salesman as an application in which a genetic algorithm proves

useful. He then notes: “Genetic Algorithms are excellent methods for hard optimization problems, where classic optimization methods fail due to difficult characteristics. Such conditions can be unsteadiness, non-derivability, noise and many others.” (2017 p 4).

Indeed, scheduling is a difficult problem to quantify. With a massive non-derivative solution space there is essentially nothing we can do to manually predict what the possible “best” schedule would look like. A general-solution program would be limited to guessing-and-checking.

If we are able to quantify the quality of a schedule, then we can optimize it with a genetic algorithm. For this experiment, we are going to attempt just that.

2. Design

Overview

We desire to create an optimal university schedule. That is, we have a set of **courses**, **professors**, **rooms**, and **timeslots (or periods)** which we must combine in a valid and preferred way. Note: we define “course” to be synonymous with “section,” meaning there may be “two” CS100s; however they each have unique identifiers within our data and may be taught by different professors (in some cases).

The simplest way to understand the relationship between all these is that *professors* and *classes* are paired, and each *timeslot* contains a copy of the full set of *rooms;* there are many rooms to one timeslot. Therefore *professor + class* pairs must be put into open *room + timeslot* combinations. For the remainder of this paper I will refer to *room + timeslot* combinations as simply *slots.*

We define the *validity* of a schedule with the following hard-requirements:

- Courses which require multimedia must be placed in a room which contains multimedia utilities.
- Courses must be placed within a room which can facilitate the entire enrolled student count for that course.
- A course can only be placed on the schedule once.
- A professor cannot teach multiple courses in the same timeslot.

In order for our algorithm to run at all, the input data must have at least one combination which can satisfy all of the above criterion. A schedule that violates these rules should not be able to be created.

We define the *value* of a schedule with the following soft-requirements:

- Delays between classes greater than two hours are not preferable.
- An instance of a course which has less students enrolled than there are seats in a paired room is not preferable.
- Scheduling a professor to teach for more than two hours at a time is not preferable.

- Scheduling a professor to teach in the same room for multiple courses in a row (regardless of whether they have a break in-between) is preferable.

The optimal schedule will maximize the above criterion while still remaining valid.

Mechanisms

The genetic algorithm relies on two scheduling mechanisms: *fill* and *swap*.

Fill builds the initial population and facilitates mutation. It takes a list of courses and guarantees every course in that list gets placed in the schedule. Fill first looks for open room slots. If no valid open slot exists, it starts displacing courses that are already scheduled. Displaced courses are then re-filled the same way, which can cascade. If fill exhausts every room and timeslot combination without finding a valid placement (for example, A displaces B, B displaces C, and C displaces A) the program will throw an error.

Swap facilitates crossover. It takes a list of timeslots and forces them into the schedule, overwriting whatever currently occupies those slots. Timeslots are keyed by time, so swapping in a new 9 AM timeslot discards the existing 9 AM timeslot entirely. The incoming timeslot's courses win any hard-requirement conflict. Courses elsewhere in the schedule that now conflict are removed and re-filled. Swap is therefore greedy.

Both mechanisms function randomly, there is no order with respect to timeslots or rooms.

Scoring

We score each schedule to denote how desirable of a solution it is.

Each schedule has a fitness score of 0 upon initialization.

In order to score our schedules we must quantify our soft-requirements:

- Delays between classes greater than 2 hours are not preferable: *subtract 15 points for each delay of 3 or more hours between courses.*
- Rooms which are scheduled under-capacity are not preferable: *subtract 1 point for each seat that is not taken during a class.*
- Scheduling a professor to teach for more than two hours at a time is not preferable: *subtract 10 points for each instance a professor has 3 or more courses in a row.*
- Scheduling a professor to teach in the same room for multiple courses in a row (regardless of whether they have a break in-between classes) is preferable: *Add 5 points for each instance that a room is duplicated past the first room for a professor.*

Our scoring logic examines a schedule and uses the criteron above to produce a fitness scores.

Selection

The project implements two selection methods, *tournament* and *elite* selection. *Tournament* selection picks two random schedules from the current population and chooses the one with the best fitness score. *Elite* chooses a random schedule from the top 20% of the population.

These selection methods were chosen because they contrast from one another. While tournament selection has a chance of choosing lower-scoring members of the population, elite selection never does. Elite selection theoretically has a chance to stagnate the fitness of the entire population, whereas tournament never does. These two selection methods contrast nicely, which is why they were chosen.

Crossover

The hyper parameter Pc controls the crossover rate. Crossover happens when building the next generation. If crossover does not occur, one of the selected parents is cloned into the next generation. If it does occur, two parents produce a child that inherits some of its periods from parent A and the rest from parent B.

Internally, a random cut point is chosen in the list of timeslots. The child is cloned from parent A and parent B's timeslots are swapped in after the random point. In this way, to some extent, the child has A's timeslots before the point and B's timeslots after.

Because crossover uses swap, it is greedy: the incoming timeslots are placed unconditionally. This preserves the integrity of the swapped genes while, as a side effect, warps the child's "original" genes.

This method of crossover was chosen over alternatives, such as a best-effort crossover which never displaces, in order to guarantee that each child produced with crossover has the correct proportion of each parent.

Mutation

The rate of mutation is controlled by hyper parameter Pm. Mutation simply selects a random course within the schedule, removes it, and re-fills it at a random location. This is done to stay accurate to how mutation occurs in nature.

Algorithm Summary

The genetic algorithm begins by using fill to randomly create the initial population. Then, it selects two parents, tries to perform crossover, tries to mutate, until it has generated a new generation.

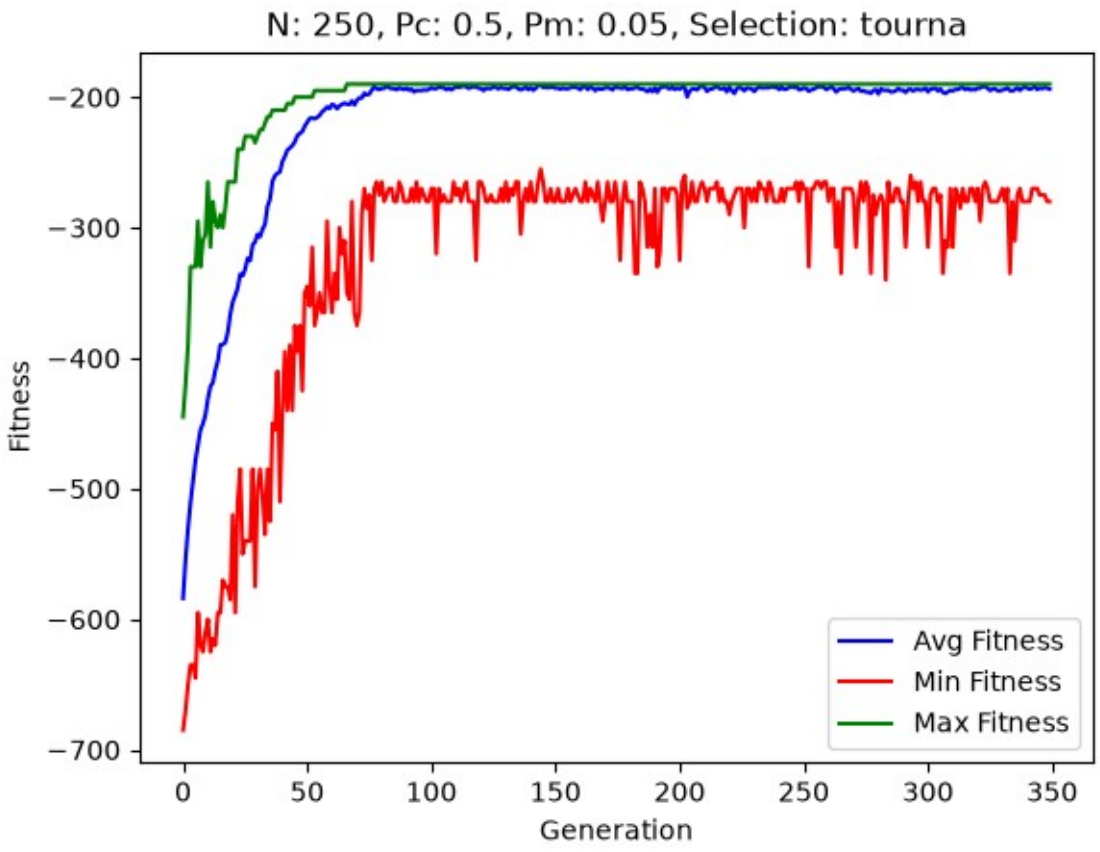
This process repeats until the number of generations produced elapses the bound set by the max generations hyper parameter.

Using the selection methods we defined, we typically choose parents with preferable traits. We then combine these traits by performing crossover, and lastly we try to generate new preferable traits by performing mutation.

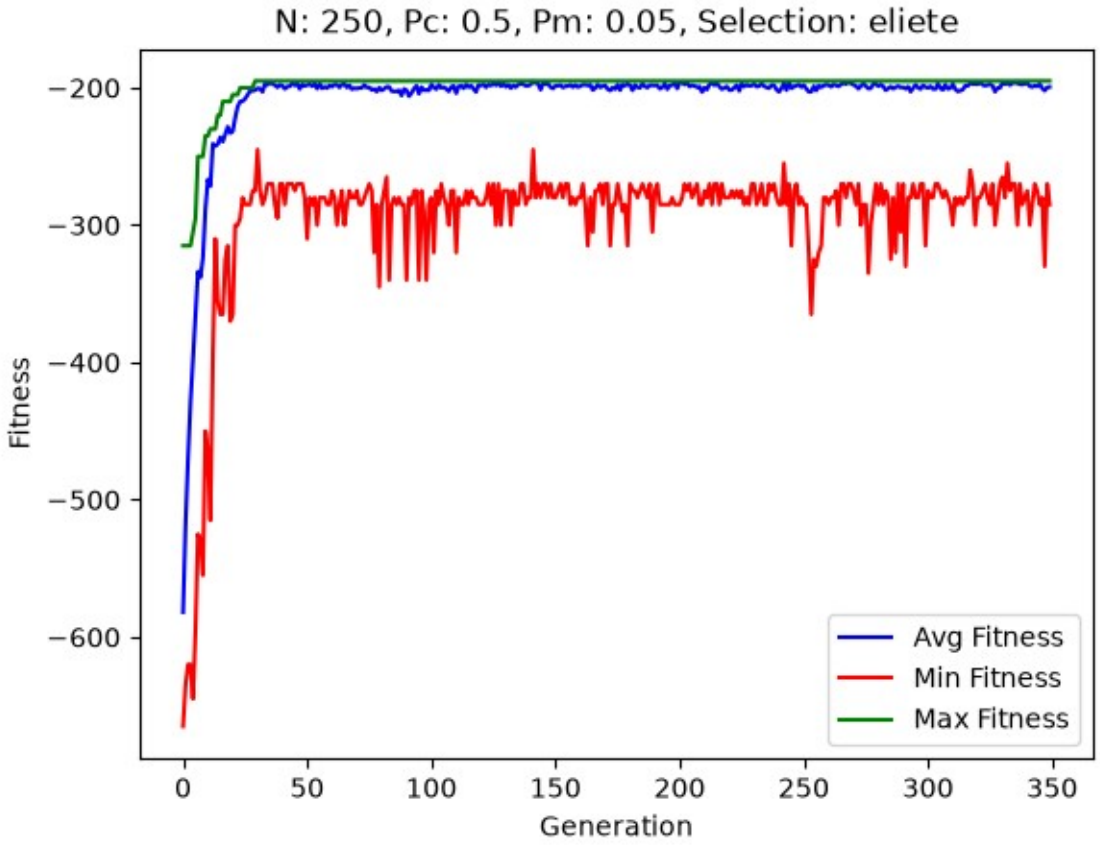
3. Results

Results set 1: Pc = 50%, Pm = 5%

Tournament Selection:



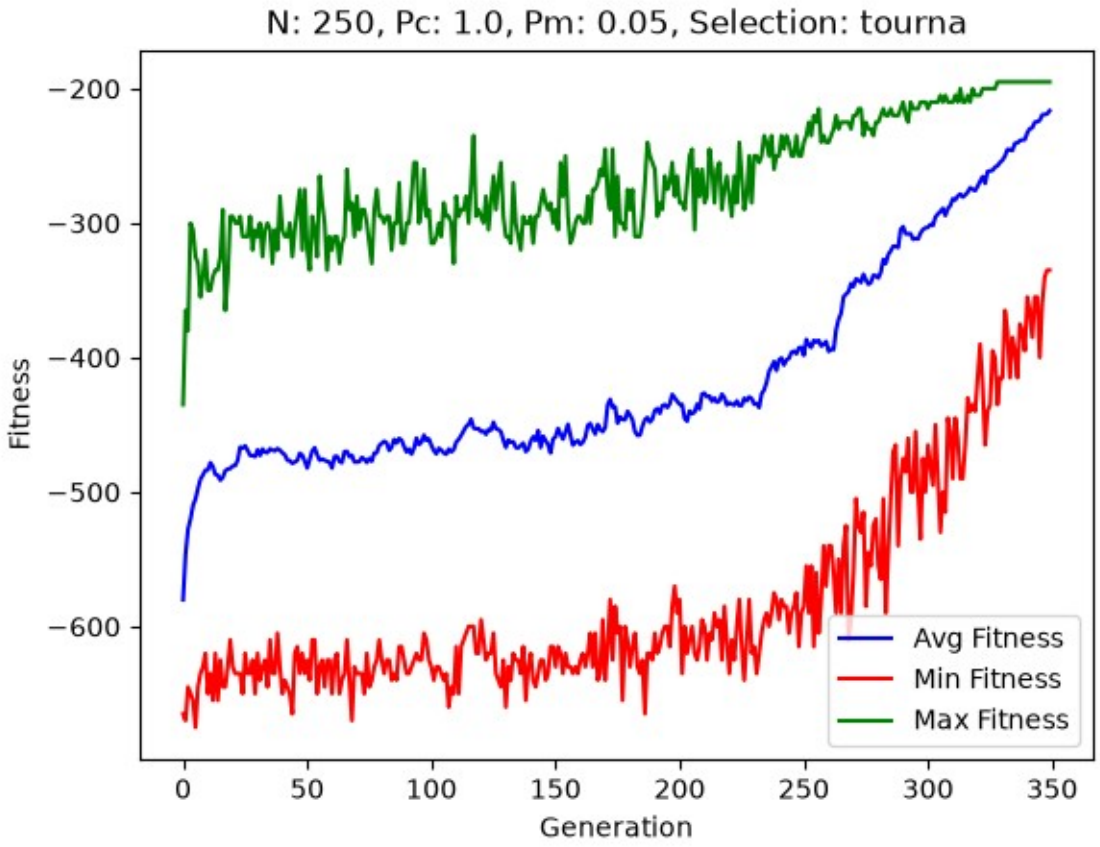
Elite Selection:


N: 250, Pc: 0.5, Pm: 0.05, Selection: eliete
Fitness
−200
−300
−400
−500
−600
0
50
100
150
200
250
300
350
Generation
Avg Fitness
Min Fitness
Max Fitness

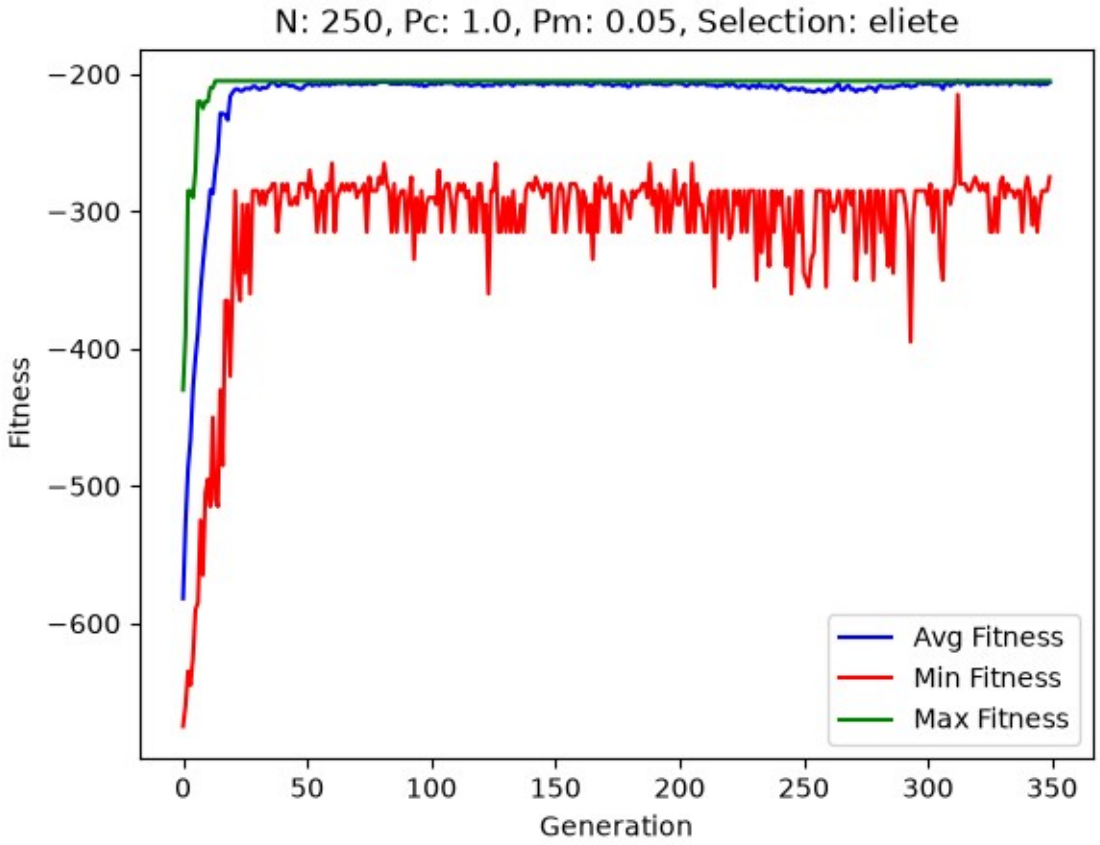
These results are pretty milk-toast. With pretty average values for crossover and mutation, elite selection finds the optimum a bit sooner.

Results set 2: Pc = 100%, Pm = 5%

Tournament Selection:



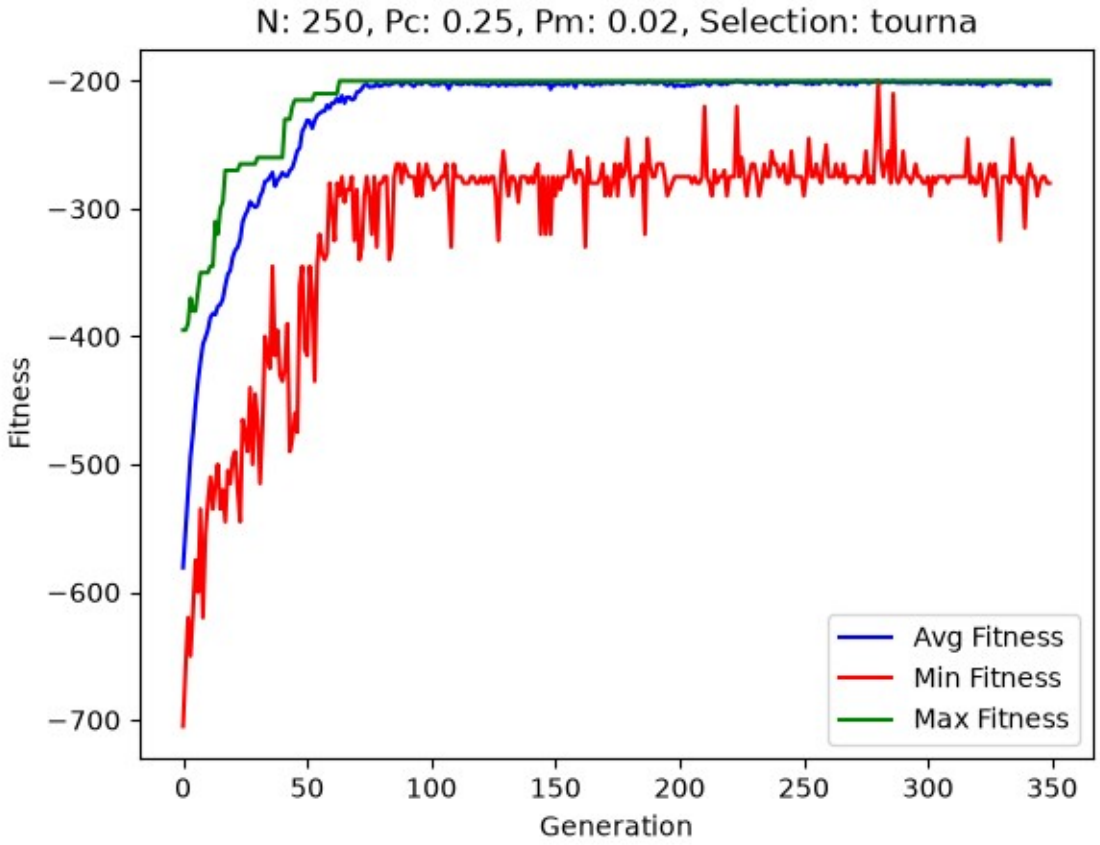
Elite Selection:



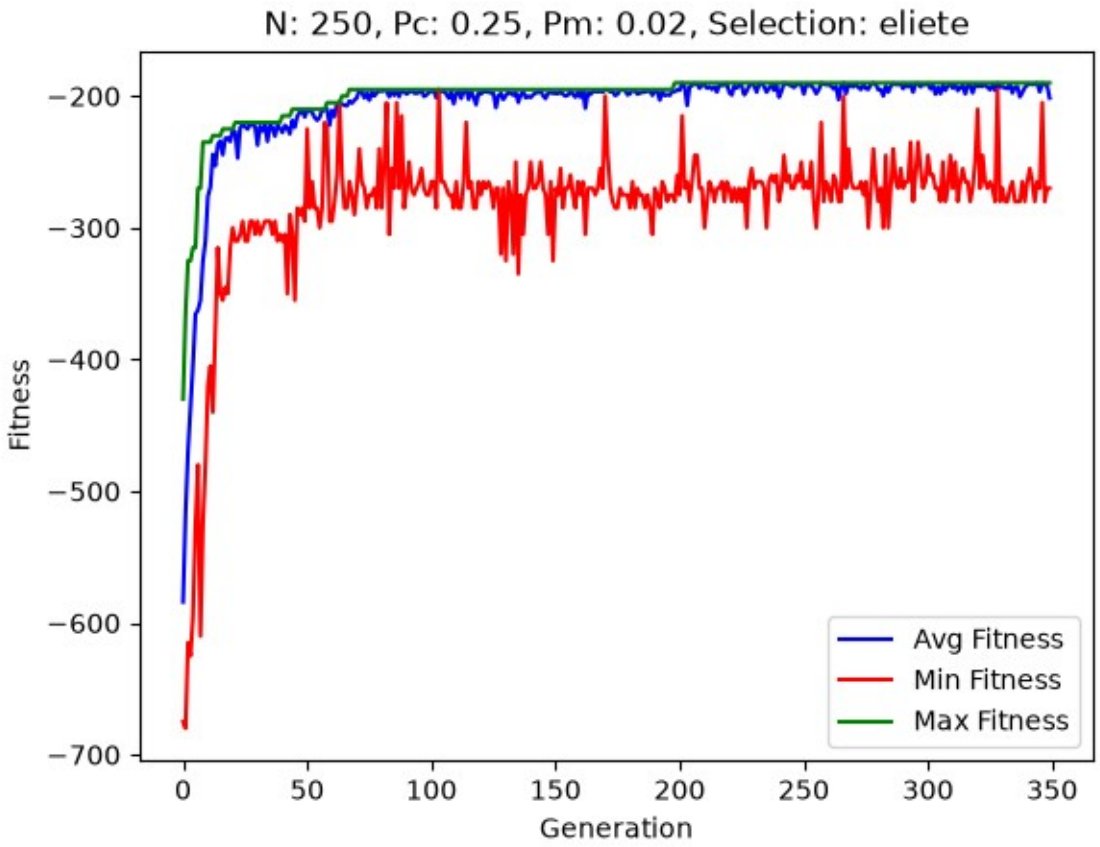
In these results, we can see that high crossover clashes with tournament selection. Elite selection is not as affected by high crossover.

Results set 3: Pc = 25%, Pm = 2%

Tournament Selection:



Elite Selection:



These results grow quickly initially but take a little longer to reach the optimal solution.

Analysis

In all these results, it appears that elite selection is superior. In every situation it finds the optimal solution as fast or faster than tournament selection, which struggles when crossover is too high.

We can see clearly the effect of greedy crossover in the 2nd result set, the tournament selection run barely finds the optimal solution before time is up. This is of course do to the fact that sub-optimal schedules are selected through tournament selection and then their timeslots are greedily inserted into a schedule which could potentially have a better fitness score. This tarnishes most of the good genes. It is only once the low-end of scores rise enough that we are able to find the solution.

Elite selection failed to result in stagnation, unlike what I was expecting. I believe this is again due to the nature of greedy crossover and the fact that it produces a form of mutation as a side effect. This shows since the best run occurred when crossover was 100% and we did elite selection.

It would also appear that lower mutation has a detrimental effect on performance, though it is a minor one. Runs with 5% mutation were able to find the last few genes needed for the optimum faster than those with 2% mutation.

4. Conclusion

Genetic algorithms are excellent in problems of exponential computation and difficult to specify quantity. In this implementation, it appears using elite selection with a high (< 50%) rate of crossover and mutation of 5% is the way to go.

My implementation is crippled by inefficiency. Having to potentially perform a full-schedule scan twice each time I want to insert a course adds up considering courses are constantly being moved. A better solution should try to index on this data rather than iterate.

It may also be a good idea to experiment further with different selection methods, or potentially even different crossover methods. For example, a selection method which only chooses parents which can undergo crossover without displacing either of one another’s courses. Or, a crossover method which is best-effort and doesn’t displace any courses.

References:

Kramer O. (2017) *Genetic Algorithm Essentials:* Springer International Publishing AG